

XM · CLASE 2

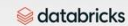


Soluciones de Analítica con Databricks

Clase 2: Arquitectura Medallion y patrones analíticos

Docente: Manuela Larrea Gómez

Agenda de la clase 2



Cerramos el laboratorio 1 y diseñamos la arquitectura de la solución

01

Lab 1 al día: primer activo gobernado

Conexión (6 pasos) · notebook (8 pasos) · verificar() · commit y PR. Guiado con lista de chequeo

02

Teoría: Medallion y contratos por capa

Bronce inmutable · Plata en formato ancho · Oro con consumidor · patrones batch/streaming/ML/GenAI · anti-patrones · caso EDP E-REDES

03

Lab 2: explorar para diseñar

Notebook de exploración · docs/architecture.md con las cinco tablas · ADR-001 granularidad y horizonte

04

Cierre

Tag s02 de la solución · así sería en Azure · próxima clase: Unity Catalog

Lab 1 al día (1/3): conexión en seis pasos

Las invitaciones ya funcionan. Si un paso ya está hecho, salta al siguiente

#	Paso	Dónde	Sé que está hecho si...
1	Cuenta en GitHub	github.com	Puedo entrar a github.com con mi usuario
2	Fork del repo + habilitar Actions	github.com/<docente>/Databricks-to-XM → Fork; en el fork: Actions → Enable	Existe github.com/<yo>/Databricks-to-XM y la pestaña Actions muestra el workflow ci
3	Entrar al workspace del curso	Enlace del chat, con el correo de la invitación	Veo el catálogo workspace en Catalog
4	Vincular GitHub en Databricks	Avatar → Settings → Linked accounts → GitHub → Link Git account	Linked accounts muestra GitHub vinculado
5	Git folder con la URL de MI fork	Workspace → Users → mi correo → Create → Git folder	Aparece Databricks-to-XM con clases/, src/, data/
6	Esquema, volumen y Excel	SQL Editor: CREATE SCHEMA / CREATE VOLUME; Catalog → raw → Upload	Catalog → workspace → c01_<usuario> → raw contiene DemandaPerdidas.xlsx

Guía completa con pantallas: clases/c01-databricks-lakehouse/slides/lab1_pasos_previos.png y lab.md

Lab 1 al día (2/3): el notebook, paso a paso

clases/c01-databricks-lakehouse/notebooks/lab01_primera_tabla_delta · widget usuario = el de tu esquema

1 Bronze: leer el Excel y escribir la tabla Delta

pandas → Spark → `_ingested_at`, `_source_file` →
`workspace.c01_<usuario>.demanda_raw`

2 Verificación: contar y consultar

Filas totales, filas por `CodigoVariable`, series activas, rango de fechas

3 Historial: escribir por segunda vez

Vuelve a ejecutar la celda `df.write...` y luego `DESCRIBE HISTORY`

4 Time travel

`SELECT count(*) ... VERSION AS OF 0`

Métrica	Valor esperado
<code>pdf.shape</code>	(145408, 8)
Filas	145.408
Por <code>CodigoVariable</code>	72.704 y 72.704
Rango de Fecha	2026-01-01 → 2026-07-25
Series activas	355
<code>DESCRIBE HISTORY</code>	≥ 2 versiones
<code>VERSION AS OF 0</code>	145.408 filas

Si un valor no coincide, no sigas: casi siempre el widget usuario no coincide con el esquema, o el Excel quedó fuera del volumen.

Lab 1 al día (3/3): verificar, commit y pull request

Cuatro OK = terminaste. Tienes hasta el minuto 35

```

verificar()
OK      Tabla con 145.408 filas
OK      Columnas _ingested_at y _source_file
OK      355 series activas
OK      Al menos 2 versiones en el historial

Listo: commit y push.

```

5

Commit & Push desde el Git folder

Botón Git → mensaje feat(01): primera tabla Delta de demanda
→ Commit & Push

6

Pull request en tu fork

GitHub → Compare & pull request → base: main de TU fork → CI
en verde → Merge

Síntoma	Causa y solución
FALTA en filas o series	Widget usuario distinto del esquema; corrige y Run all
No such file al leer el Excel	Quedó en Workspace, no en el volumen. Súbelo desde Catalog → raw
ModuleNotFoundError: openpyxl	Ejecuta la primera celda (%pip install) y reinicia
FALTA en versiones	Solo escribiste una vez: repite la celda df.write...
Push rechazado	GitHub no vinculado (Settings → Linked accounts) o Git folder apunta al repo original, no a tu fork
PR sin checks	Actions no habilitado en el fork: pestaña Actions → Enable

ARQUITECTURA

Medallion: capas, contratos y patrones

Ya tienes una tabla con datos crudos. ¿Y ahora qué?

Medallion: tres capas de calidad creciente, en un solo sentido

Cada capa se reconstruye desde la anterior. Oro nunca escribe en plata; plata nunca escribe en bronce



Si cambia una regla de negocio: se borra plata y se recalcula desde bronce. Sin volver a pedir el archivo.

Un contrato por capa: cinco cosas que cada tabla promete

Esto es lo que vas a llenar hoy por tabla en docs/architecture.md

	Grano (qué es una fila)	Quién escribe	Quién lee	Frescura	Qué garantiza
Bronce	Un registro publicado	Ingesta (Auto Loader / job)	Ingeniería de datos	Cuando llega un archivo	Nada se perdió, nada se modificó
Plata	Una serie en un día	Pipeline declarativo	Analistas, modelos	Diaria, tras bronce	Pasó las reglas de calidad; una sola versión vigente
Oro	Lo que el consumidor necesita	Job de features / inferencia	Negocio, tablero, app	Pactada con el consumidor	Listo para usar sin transformar

Si no sabes quién es el dueño de una tabla, ese es el problema que hay que resolver antes de escribir código. Pregunta útil: ¿quién reclama si esta tabla llega tarde?

Bronce: inmutable y con metadatos. La razón está en tu propio dato

Cada Fecha tiene hoy una sola FechaPublicacion; el patrón de publicación dice lo que viene

FechaPublicacion	Días publicados	N
2026-05-06	todo febrero	28
2026-06-20	todo abril	30
2026-07-07	marzo + 1–2 jul	32
2026-07-10	mayo + 3–5 jul	33
2026-07-11	junio + 6 jul	31
2026-07-25	1 ene → 20 jul	32
julio (diarios)	un día cada uno	1

Retraso entre el día del dato y su publicación: mínimo 5 días, mediana 68, máximo 205.



Regla 1: bronce es append

Nunca update, nunca delete. Si el mismo día llega otra vez, se agrega otra fila con otra fecha de publicación.



Regla 2: metadatos obligatorios

`_ingested_at` (cuándo entró), `_source_file` (de qué archivo), `_publication_date` (la fecha de publicación que trae el dato).



Consecuencia

Plata decide cuál versión es la vigente (clase 4). Bronce solo acumula evidencia. Así se explica por qué el pronóstico de junio usó otro número.

El 25 de julio XM volvió a publicar medio año. En producción, el archivo de la próxima semana traerá días que ya tenían valor.

Plata: de reporte a tabla analítica

Formato largo → formato ancho, una fila por serie-día, con reglas de negocio aplicadas

Fecha	Serie	CodigoVariable	Valor
2026-05-28	EMED · POPAYAN · NR · EDUC.	DdaReal	5836,69
2026-05-28	EMED · POPAYAN · NR · EDUC.	PerdidasEnergia	77,96

Formato largo: dos filas por serie-día (así llega)



fecha	serie	demanda_real_kwh	perdidas_kwh	fecha_pub
2026-05-28	EMED · POPAYAN · NR · EDUC.	5836,69	77,96	2026-07-10

Formato ancho: una fila por serie-día (así se analiza)



Reglas que el dato ya sugiere

$\text{perdidas_kwh} \leq \text{demanda_real_kwh}$ (se cumple en el 100 %) · Regulado ⇒ CIU = SIN CLASIFICAR (se cumple) · valores ≥ 0



Normalización

snake_case sin tildes · tipos DATE y DOUBLE · CIU contra dim_ciu · llave de serie explícita



Versión vigente

fecha_publicacion_vigente: de qué publicación viene el valor. Historial completo queda en bronce



Dimensión CIU

codigo, seccion, nombre. Pequeña, pero une con cualquier fuente que hable de sectores

silver_energia.demanda_diaria · silver_energia.dim_ciu — se construyen en las clases 5 y 6

Oro: dos tablas, cada una con un consumidor con nombre

Las dos llevan versión del modelo y de la corrida: eso es linaje modelo ↔ datos

<code>gold_energia.features_demanda_diaria</code>	
Grano	serie-día
Columnas	las de plata + lag_1, lag_7, lag_14, lag_28, media_7, media_28, dia_semana, festivo
Consumidor	entrenamiento e inferencia (clases 9–11)
Frescura	diaria, después de plata
Garantiza	sin fuga de futuro: cada feature usa solo datos ≤ fecha

<code>gold_energia.pronostico_demanda</code>	
Grano	serie · fecha_pronostico · horizonte (1–7)
Columnas	demanda_estimada_kwh, modelo_version, run_id, fecha_ejecucion
Consumidor	negocio: tablero AI/BI y app (clase 11)
Frescura	diaria, tras la inferencia
Garantiza	se puede reconstruir qué modelo produjo cada número

Más adelante (clase 15) aparece una tercera: `gold_energia.desempeno_modelo`, observado vs. estimado. Hoy no hace falta diseñarla.

Patrones de solución: cómo fluyen los datos

Medallion organiza; el patrón define latencia, cómputo y complejidad



Batch

Lotes en horario. Archivos, extractos, cierres. Latencia de horas o días. El más barato y el más común.

Lakeflow Jobs, Auto Loader



Streaming

Eventos continuos, latencia de segundos. Medidores inteligentes, telemetría de subestaciones, SCADA.

Structured Streaming, Declarative Pipelines



Machine learning

Sobre batch o streaming: entrena con histórico, infiere sobre lo nuevo, monitorea y reentrena.

MLflow, Model Registry, Serving



GenAI y agentes

Modelos de lenguaje sobre documentos o datos: RAG, agentes, Genie. Está en la plataforma; no lo construimos.

Mosaic AI, Agent Bricks, Vector Search

El nuestro: batch diario + ML. Los datos llegan por archivo con días de retraso y el pronóstico se necesita una vez al día. Decirlo explícitamente es una decisión de arquitectura: evita montar streaming porque suena mejor.

Anti-patrones al migrar notebooks a Databricks

Lo que sale mal cuando se cambia de herramienta sin cambiar de hábitos, y la práctica que ya está en el repo

Anti-patrón	Por qué duele	Práctica en el repo
Un notebook que lee, limpia, entrena y escribe en una celda de 300 líneas	No se puede probar ni reutilizar; nadie más lo entiende	Notebooks delgados; lógica en src/xm_demanda con pytest
Rutas y nombres de tabla escritos a mano	Funciona en dev y se rompe en prod	conf/<env>.yaml + variables del bundle
Limpiar al cargar (en bronce)	Se pierde la evidencia y no se puede reprocesar	Bronce append con metadatos; la limpieza vive en plata
Una tabla por notebook por persona	Cinco versiones de la verdad	Un esquema por capa y dominio; una tabla plata compartida
Sin metadatos de carga	Nadie sabe qué archivo produjo qué fila	_ingested_at, _source_file, _publication_date obligatorios
Sin contrato de frescura	Negocio pregunta si el dato es de hoy y nadie sabe	Frescura y dueño escritos en docs/architecture.md

Sector: EDP E-REDES pronostica 200.000 curvas de carga cada día

Proyecto PREDIS, operador de distribución de Portugal, sobre Databricks



Lo que hacen

~200.000 diagramas de carga diarios de instalaciones de media y alta tensión · 96 puntos de 15 min por curva · 60 millones de registros al día · Spark + pandas UDFs para entrenar en paralelo · Workflows separados para entrenamiento e inferencia · ensamble de Elastic Net, LightGBM, Prophet y baselines; cada día eligen por serie el que mejor lo hizo ayer · inferencia en ~3 h sobre un clúster de 40 nodos.

De su patrón...	...a nuestras 355 series
Capas bronce → plata → oro	Idéntico
Muchas series, un modelo global o ensamble	Idéntico, a escala diaria
Job de entrenamiento ≠ job de inferencia	Idéntico (clases 10–12)
Selección diaria del mejor modelo por serie	Backlog post-curso
Clúster de 40 nodos, 3 h de inferencia	Serverless, minutos
Datos de 15 minutos	Datos diarios; el patrón no cambia

Fuente: Databricks Community, "Efficient distributed energy load forecasting with Databricks at EDP E-REDES". Enlace en [lecturas.md](#).

Lo que el dato ya nos dijo: seis hechos, seis decisiones pendientes

Las vas a reproducir en el notebook del lab. No las respondas ahora: anótalas para el documento y la ADR

Hecho	Cifra	Decisión de diseño que abre
Formato largo	2 filas por serie-día; 72.704 pares	Plata en formato ancho: ¿qué pasa si falta una de las dos filas?
Publicación con retraso variable	5 a 205 días; lotes mensuales; republicación de 6 meses el 25-jul	Bronce append con <code>_publication_date</code> ; plata elige la vigente
Series incompletas	4 de 355; una con solo 7 días	¿Rellenar con cero, marcar inactiva o excluir del modelo?
Regulado vs. no regulado	29 series = 69 % de la energía; 326 series pequeñas y ruidosas	¿Un modelo global, dos, o uno por tipo de mercado?
Ceros	299 días con demanda 0, concentrados en pocas series	¿Dato real, ausencia o error? Regla de calidad en plata
Pérdidas	≈1,5 % de la demanda; nunca mayores	Regla $\text{perdidas} \leq \text{demanda}$; ¿pronosticar pérdidas o solo demanda?

Laboratorio 2 (1/2): explorar para diseñar

clases/c02-arquitectura-medallion/notebooks/lab02_explorar_para_disenar · widget usuario

1 Abrir el notebook y apuntar a tu tabla

Widget usuario = el de tu esquema. Si no terminaste el lab 1: usuario = docente (tabla compartida en solo lectura)

2 Reproducir los seis hechos

Formato largo · patrón de publicación · series incompletas · regulado vs. no regulado · ceros · pérdidas. Cada celda termina con una pregunta

3 Anotar respuestas

En la celda de texto al final de cada bloque. Son el insumo del documento de arquitectura

```
lab02 · fragmento

-- ¿cuántas filas por serie-día?
SELECT n, count(*) FROM (
  SELECT Fecha, <llave>, count(*) n
  FROM demanda_raw GROUP BY ALL
) GROUP BY n;

-- retraso de publicación
SELECT min(d), median(d), max(d)
FROM (SELECT datediff(
  FechaPublicacion, Fecha) d
  FROM demanda_raw);

-- resultado esperado: 5 · 68 · 205
```

No hay que escribir código nuevo: hay que leer resultados y decidir.

Lab 2 (2/2): architecture.md y ADR-001

25 min · en tu Git folder, rama feature/c02-arquitectura

4

Llenar docs/architecture.md

Cinco tablas: bronce demanda_raw, plata demanda_diaria, dim_ciu, oro features, oro pronóstico. Por cada una: grano, dueño (un rol de XM), frescura, garantías

5

Escribir docs/adr/ADR-001-granularidad-horizonte.md

Contexto · Decisión (diaria, 7 días, modelo global) · Alternativas descartadas y por qué · Consecuencias. Estado: aceptada

6

verificar() en el notebook

Revisa que las tablas no tengan celdas vacías y que la ADR esté aceptada con decisión escrita

7

Commit, push, PR

feat(c02): arquitectura y ADR-001 → PR a main de tu fork → CI verde → merge

Criterios de aceptación

- architecture.md: 5 tablas con grano, dueño, frescura y garantías
- ADR-001 en estado aceptada, con al menos 2 alternativas descartadas
- verificar() en OK; PR en verde

Hito 1 cerrado cuando...

verificar() del lab 1 en verde, architecture.md completo y ADR-001 aceptada. Al cierre se libera el tag s02: compara con git diff s02 -- docs.

Así sería en Azure Databricks: Medallion sobre ADLS Gen2 y Unity Catalog

Elemento	Hoy en Free Edition	En XM sobre Azure
Dónde viven las capas	Almacenamiento gestionado por Databricks	Contenedores o rutas de ADLS Gen2 por capa, registrados como external locations en UC
Cómo llega el archivo a bronce	Upload manual al volumen	Landing zone en ADLS; Auto Loader lo ingiere al aterrizar (clase 4)
Nombres	workspace.c01_<usuario>.demanda_raw	<env>.bronze_energia.demanda_raw según el estándar de nomenclatura de XM
Contratos	Escritos en docs/architecture.md	Además: tags y comentarios de tabla en Unity Catalog, visibles sin abrir el repo
Dueños	Un rol escrito en el documento	Grupos de Entra ID como owners de esquema y tabla
Reproceso plata desde bronce	Borrar y recalcular a mano	Full refresh del pipeline declarativo, con historial Delta intacto

Lo que no cambia: las cinco tablas, los cinco contratos y la ADR.

Resumen de la clase 2 y preparación para la clase 3

Logros de hoy

✓ Lab 1 cerrado

Primera tabla Delta, historial, commit y PR

✓ Medallion con contratos

Grano, dueño, lectores, frescura, garantías

✓ Bronce inmutable

Append + metadatos; la republicación del 25-jul lo justifica

✓ Plata y oro

Formato ancho; dos tablas oro con consumidor con nombre

✓ ADR-001

Diaria, 7 días, modelo global. Hito 1 cerrado

Próxima clase

→ Unity Catalog

Catálogos dev/qa/prod, esquemas por capa, grupos por rol

→ Permisos mínimos

Grants a grupos; un usuario sin permiso rompe un job y se diagnostica

→ Linaje y auditoría

Quién leyó qué y cuándo

→ Traer a la clase 3

architecture.md terminado: los esquemas que diseñaste son los que vas a crear



Gracias

Nos vemos en la clase 3: Unity Catalog, gobierno, seguridad y acceso

DOCENTE

Manuela Larrea Gómez